

Introduccion

En un enfoque tradicional se programaba "a ciegas" ya que la prueba se implementaba al final del ciclo, lo que generaba re trabajo debido a la rigidez con la que se implementaban las soluciones. A medida que avanza el tiempo el costo de resolver defectos se hace mas grande

TDD no es una practica de testing, es una practica de desarrollo que utiliza conceptos del testing. Guia el desarrollo con un enfoque mas proactivo

3 leyes del TDD

- No escribiras linea sin antes un test unitario que falle
- No escribiras mas codigo de prueba que el necesario para provocar un fallo
- No escribiras mas codigo de produccion que el minimo para que el test pase

Ciclo Red - Green -Refactor

Red: Escribimos la prueba que falla declarando intencion. Quiero que al pasar **estos** datos, el sistema me devuelva **este** resultado.

Green: Pasamos a un enfoque imperativo (el como), escribimos el codigo minimo para que pase la prueba, sin preocuparnos por buenas practicas ni estructura. Se puede empezar por informacion mockeada incluso (patron fake it), y de ahi pasamos a definir parametros simples. Empezar por lo mas simple para no perder el foco y poder ir creciendo la complejidad del codigo

Refactor: Modificar estructura interna sin alterar su comportamiento externo. No cambiamos logica, cualquier funcion que llamaba a la funcion debe seguir recibiendo los mismos resultados, solo mejoramos el código en términos de estructura y distribución

Cuando se completan las tres etapas se completa un ciclo y se pasa al siguiente. Partimos de soluciones particulares a soluciones genericas

Patrones

definen como pasamos de una etapa a la otra, son solo ayudas.

De red:

- Starter test -> punto de partida, la prueba mas trivial y facil para arrancar con el ciclo
- One step test -> seguimos con un flujo de complejidad de la mas facil a la mas dificil

- Explanation test -> para etapa mas madura de ciclo, se pueden generar debates sobre el comportamiento que deberia tener el codigo, estos debates los manifestamos en pruebas para solucionarlo
- Learning test -> cuando no tenemos codigo propio (librerias, apis), el aprendizaje de este codigo los planteamos en pruebas para luego integrarlo de forma efectiva al sistema
- Another test -> evitar distracciones y enfocarnos en lo principal, pueden surgir distintas pruebas alternativas que deberan solo anotarse y primero terminar con el actual
- Regression test -> cuando ocurre un problema en produccion, dejo asentado los pasos que llevaron a ese problema para luego implementarlos y replicarlos en un test

De green:

- Fake it -> harkodear retorno en una primera etapa
- Triangulacion -> armar varios casos de falla para asegurar que la falla esta en la logica y no en un caso aislado particular
- Obvious Implementation -> si ya conoces la implementacion podes saltar los pasos de solucion particular y pasar a una solucion mas generica
- One to many -> enfocarnos en unidades y posteriormente en colecciones (por ejemplo una suma puede ser de dos numeros o de un conjunto)

Diseño de test:

- Test list -> defino el listado de pruebas a realizar
- Isolated test -> los test deben ser los mas independientes posible
- Test first -> no escribo codigo hasta tener una prueba
- Assert test -> arrancar desde lo que se quiere probar hasta lo que se tiene que configurar para conseguirlo
- Test data -> info sencilla y facil de entender
- Evident data -> Relacion entre entrada y salidas debe ser lo mas obvia posible

DISEÑO DE TEST

➔ PATRONES DE TESTING

➔ PATRONES DE DISEÑO

➔ XUNIT PATTERNS

- ASSERTION
- FIXTURE
- EXTERNAL FIXTURE
- TEST METHOD
- EXCEPTION TEST
- ALLTEST



Caso Taxi Mobile

CASO TAXI MOBILE

Ver mapa de taxis

5

Como Administrador de la Central quiero ver la ubicación de todos los taxis de la central y si tienen viajes en curso para saber la disponibilidad actual

Criterios de Aceptación:

- Se muestran taxis en todos los estados (Verde: Libre, Amarillo: Solicitado, Rojo: Ocupado, Negro: Fuera de Servicio).
- Los taxis que se mostrarán deben tener habilitado el sistema de ubicación.
- Pasando el mouse sobre el ícono del auto deben poder visualizarse los datos del viaje si el taxi está ocupado (hora de inicio y costo) y los datos del pasajero (nombre, apellido y número de teléfono celular).
- Si el taxi está solicitado, sólo se visualizarán los datos del pasajero.
- Podrán utilizarse filtros por Barrio, por Estado y por chapa del taxi

Pruebas de Usuario

- Probar visualizar un conjunto de taxis libres (pasa)
- Probar visualizar un conjunto de taxis solicitados (pasa)
- Probar visualizar un conjunto de taxis fuera de servicio (pasa)
- Probar visualizar un conjunto de taxis ocupados (pasa)
- Probar visualizar un conjunto de taxis libres, solicitados, fuera de servicio y ocupados (pasa)
- Probar visualizar un taxi conectado al sistema por número de chapa (pasa)
- Probar visualizar en el mapa un taxi no conectado al sistema de ubicación (falla)
- Probar visualizar un conjunto de taxis en un Barrio inexistente (falla)



Como precondition debo tener usuarios logueados por lo que lo mockeo, lo mismo con los taxis y la ciudad

Luego llamamos a la funcion que queremos probar y lo guardamos en una variable y validamos que los datos de esa variable sean los esperados

Por ultimo definimos el codigo que hace que los test fallen

RED



```
def test_mostrar_mapa_taxis_libres_en_barrio_exito():
    //Precondiciones
    usuarios = [{user: "salvita", rol: "Administrador de la Central"}]
    loguear(salvita)
    configurar_ciudad_prueba("Córdoba Capital")
    taxis = [{'chapa': 'ABC123', 'estado': 'Libre', 'barrio': 'Nueva Córdoba', 'ubicacion': (10, 20)}]

    //Pasos
    mapa_mostrado = mostrar_mapa(filtros = ["barrio", "estado"], barrio = "General Paz", chapa = "", estado = "Libre")
    //Resultado
    assert all (mapa_mostrado.ciudad.barrio == "General Paz", "No existe tal barrio para esa ciudad ingresada")
    assert all(taxi.barrio == "General Paz" for taxi in mapa_mostrado.taxis, "No todos los taxis están dentro del area filtrada ")
    assert all(taxi.estado == "Libre" for taxi in mapa_mostrado.taxis, "No todos los taxis están libres")
    assert all(taxi.icono == "verde" for taxi in mapa_mostrado.taxis, "No todos los taxis tienen el icono verde")

def mostrar_mapa(filtros, barrio, chapa, estado):
    pass
```

Pasamos al green en donde definimos el codigo que hara que los tests pasen

GREEN



```
def test_mostrar_mapa_taxis_libres_en_barrio_exito():
    //Precondiciones
    usuarios = [{user: "salvita", rol: "Administrador de la Central"}]
    loguear(salvita)
    configurar_ciudad_prueba("Córdoba Capital")
    //cargó los datos al sistema.
    taxis = [{'chapa': 'ABC123', 'estado': 'Libre', 'barrio': 'General Paz', 'ubicacion': (10, 20)}]

    //Pasos
    mapa_mostrado = mostrar_mapa(filtros = ["barrio", "estado"], barrio = "General Paz", chapa = "", estado = "Libre")
    //Resultado
    assert all (mapa_mostrado.ciudad.barrio == "General Paz", "No existe tal barrio para esa ciudad ingresada")
    assert all(taxi.barrio == "General Paz" for taxi in mapa_mostrado.taxis, "No todos los taxis están dentro del area filtrada ")
    assert all(taxi.estado.nombre == "Libre" for taxi in mapa_mostrado.taxis, "No todos los taxis están libres")
    assert all(taxi.estado.icono == "verde" for taxi in mapa_mostrado.taxis, "No todos los taxis tienen el icono verde")

def mostrar_mapa(filtros, barrio, chapa, estado):
    return new Mapa([taxi={chapa = "ABC123", estado = {nombre = "Libre", icono = "verde"}, barrio = "General Paz"}])
```

Limpiamos codigo e implementacion, sin cambiar logica de funcionamiento

REFACTOR



```
def mostrar_mapa(filtros: String[], barrio: Barrio, chapa: String, estado: Estado) -> Mapa:
    estado_libre = new Estado(
        nombre = "Libre",
        icono = "verde"
    )

    barrio_general_paz = new Barrio(
        nombre = "General Paz"
    )

    taxi = new Taxi(
        chapa = "ABC123",
        estado = estado_libre,
        barrio = barrio_general_paz
    )

    taxis = [taxi]

    return new Mapa(taxis = taxis)
```

Posteriormente en otros ciclos se implementaran validaciones, llamadas a bases de datos y otras funcionalidades, pero en un primer ciclo esto ya funciona

EJEMPLO FINAL

```
def mostrar_mapa(filtros: Filtros, barrio: Barrio, chapa: String, estado: Estado) -> Mapa:
```

```
    // Caso de prueba taxis desconectados
```

```
    if (filtros == 'Mostrar todos los taxis')
        taxis = buscarTaxiEnBarrio(barrio)
        (ubicación == None) //ubicación desconectada
        taxis = buscarTaxiDesconectado(ubicación)
```

```
    // Caso de prueba mostrar taxis en distintos estados en un barrio
```

```
    if (filtros == 'Mostrar taxis por estado y por barrio')
        taxis = buscarTaxiPorEstadoYPorBarrio(barrio, estado)
```

```
    // Caso de prueba mostrar taxis por chapa
```

```
    if (filtros == 'Mostrar taxis por chapa')
        taxis = buscarTaxiPorChapa(chapa)
```

```
    // Caso de prueba mostrar taxis por barrio y datos de viaje
```

```
    if (filtros == 'Mostrar taxis por barrio')
        taxisFiltrados = buscarTaxiEnBarrio(barrio)
        //foreach simplificado
        if (taxis[i].getEstado() == 'Solicitado' && filtros == 'Mostrar datos viaje')
            taxis =
                taxis[i].getChapa()
                taxis[i].getViajeActual().getPasajero().getNombre()
                taxis[i].getViajeActual().getPasajero().getTelefono()
                taxis[i].getViajeActual().getFechaYHora()
```

```
    return new Mapa(taxis=taxis)
```

Pruebas de Usuario

- Probar visualizar un conjunto de taxis libres (pasa)
- Probar visualizar un conjunto de taxis solicitados (pasa)
- Probar visualizar un conjunto de taxis fuera de servicio (pasa)
- Probar visualizar un conjunto de taxis ocupados (pasa)
- Probar visualizar un conjunto de taxis libres, solicitados, fuera de servicio y ocupados (pasa)
- Probar visualizar un taxi conectado al sistema por número de chapa (pasa)
- Probar visualizar en el mapa un taxi no conectado al sistema de ubicación (falla)
- Probar visualizar un conjunto de taxis en un Barrio inexistente (falla)

